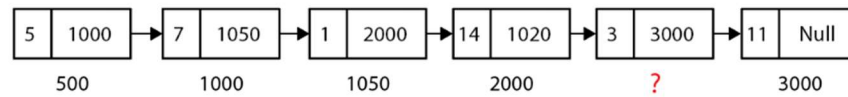


Module-5



- Why do you think linked-list requires more memory than an array when storing the same number of elements?
- Write down Three Limitations of the array which can be solved by the use of Linked List
- What is the value of Head?
- What is the value of ? marked address location?
- What will be the value of Head->Next->Next->Value?
- What will be the value of **Sum** following pseudocode snippets?

```
Sum = 0
Temp = Head
While ( Temp -> Next != 1020 ) {
    Sum += Temp-> value
    Temp = Temp -> Next
}
Sum -= Temp -> value;
```

Solution:

(a)

Memory Overhead: Each node in a linked list contains two parts: the data and the pointer (or reference) to the next node. This pointer requires additional memory.

Non-contiguous Allocation: Arrays store elements in contiguous memory locations, which makes them more memory-efficient for fixed-size data structures.

Pointer Storage: In contrast, a linked list requires extra memory to store the addresses (pointers) of subsequent nodes, leading to higher memory usage.

(b)

❓ **Fixed Size:**

- Arrays have a fixed size determined at the time of declaration, making it difficult to handle dynamically growing data.
- Linked lists can grow or shrink dynamically as nodes are added or removed.

❓ **Insertion/Deletion Complexity:**

- Inserting or deleting elements in an array is inefficient because it often involves shifting elements.
- Linked lists allow efficient insertion and deletion by adjusting pointers.

❓ **Wasted Memory:**

- If the array size is overestimated, unused memory is wasted.
- Linked lists allocate memory only for the elements currently present.

(C)

The Head points to the first node in the list, which has the address 500.

(D)

The value of ? marked address location 1020.

(e)

- ❓ Start at Head (500), which points to the first node (5).
- ❓ Head->Next moves to 1000, which contains the second node (7).
- ❓ Head->Next->Next moves to 1050, which contains the third node (1).
- ❓ The value at Head->Next->Next->Value is 1.

(f)

Step-by-Step Execution:

- 1. Initialization:**
 - Sum = 0.
 - Temp = Head (address 500, value 5).
- 2. Iteration 1:**
 - Temp->Next = 1000 $\neq$ 1020.
 - Add Temp->Value (5) to Sum.
 - Sum = 5.
 - Move Temp to 1000.
- 3. Iteration 2:**
 - Temp->Next = 1050 $\neq$ 1020.
 - Add Temp->Value (7) to Sum.
 - Sum = 12.
 - Move Temp to 1050.
- 4. Iteration 3:**
 - Temp->Next = 2000 $\neq$ 1020.
 - Add Temp->Value (1) to Sum.
 - Sum = 13.
 - Move Temp to 2000.
- 5. Iteration 4:**
 - Temp->Next = 1020 (loop condition fails here).
- 6. Final Step:**
 - Subtract Temp->Value (14) from Sum.
 - Sum = 13 - 14 = -1.